

I just recently read the book « Design patterns: Elements Of Reusable Object-Oriented Software » by the gang of four. And while the code was hard to understand, it really is a reference book that I can't recommend enough. That is why I would like to share what I took from it as well as how it helped me improve my coding skills and most importantly what I did to remember most of the designs I saw.

Why do we need design patterns?

Well for a lot of you, there is an obvious answer to this. But for beginners like myself, coding was all about sitting down and actually getting things done whenever you have a problem to solve or an application to build. I thought that only inexperienced programmers would take the time to sit down and plan every single step (the senior developers; the know-it-alls!).

But what led me to design patterns and then to this wonderful book in the first place are the bigger projects that needed to be flexible and were intended to stay in the long run. That is when I realised that this naive way of approaching programming wasn't helping me because I would end up each time with a big pile of mess that is impossible to understand (even for me), extend, or maintain.

How to remember design patterns

First, I would suggest you learn the SOLID principles before diving into the design patterns, as they are the basis of it, and focus more on the important ones that are frequently used.

For those of you who read this particular book or any other regarding this domain, you will know that there are a lot of them! Each one of those principles solves a particular problem. Some of them are even interchangeable and others work together. And it could be hard for beginners to remember and use them when needed.

Originally, I tried to remember the book's definition of each one of them, and while some of them truly capture the meaning like for the singleton pattern "Ensure a class only has one instance, and provide a global point of access to it.". Others, I find, are just not as expressive as you would expect.

But due to the importance of this, I had to come up with a way to keep them available in my toolkit at any time. What I did was I linked them with everyday life situations and actions. Here is a list of some of the book's design patterns, their definition, and what made me remember them.

PS: These are just to make the intent of each design pattern seem more intuitive, the implementation should be understood. And you are welcome to change them to fit a more relatable situation that you encounter frequently.

1-Creational design pattern

- **Abstract factory design pattern**

“Provide an interface for creating families of related or dependent objects without specifying their concrete classes.”

If you buy a keyboard, you're most likely to buy a mouse with it. Therefore the E-commerce websites can suggest buying the two at once. I say they are using an abstract factory. Forcing you to buy related objects together.

- **Builder design pattern**

“Separate the construction of a complex object from its representation so that the same construction process can create different representations. ”

If the process to build a desk or any furniture is hard and tedious, I say that the company needs to create a builder to simplify the process. By giving you a friendly manual with step-by-step instructions for you to follow. Or it could even send an employee (builder) that does the construction from scratch.

- **Factory design pattern**

“Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses. ”

The name itself gives enough information, just think of it as a real factory.

- **Singleton design pattern**

“Ensure a class only has one instance, and provide a global point of access to it.”

Call a singleton an object that you configure or turn on only once, like setting your game graphics setting. And then you or everyone else can use it directly without having to waste time in the future.

2-Structural design patterns

- **Adapter design pattern**

“Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces. ”

We've all used real adapters at some point when we couldn't directly connect two devices together.

- **Composite design pattern**

“Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly. ”

You wouldn't talk to your boss the same way you'll talk to your work colleague due to the hierarchy. But it would be easier if you could send the same email to both of them without having to change anything. If only you could use the composite pattern.

- **Decorator design pattern**

“Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.”

Normally decorating something won't add any functionality to it. But think of it as a superhero suit that gives more power to the person who wears it (laser beams eyes, super strength, etc...)

- **Proxy design pattern**

“Provide a surrogate or placeholder for another object to control access to it.”

When you're a child, your proxy is by definition a person that has some sort of power over you like your parents. They control how and when people talk to you, in what circumstances and they can basically make decisions for you. But all of it is essential to protect you.

3-Behavioral design patterns

- **Command design pattern**

“Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.”

Lately, a lot of tech devices like mice or smart lights give you a customizable button. This provides customers with more flexibility since the command is not directly attached to the button.

- **Iterator design pattern**

“Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.”

When you're using your favorite social network, you keep scrolling down and up the list of posts. You are going through the list without knowing which algorithm is used to iterate through the posts or what exact type of data you are scrolling. You're scrolling down the window regardless of what is happening in the background. This is what the iterator pattern is for.

- **Strategy design pattern**

“Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.”

When you buy a pencil you usually don't care what brand it is as long as it's working and it's not expensive. This means that they are interchangeable for you and when you go to the market, you look for a pencil, not a particular brand. This is the strategy pattern.

- **Observer design pattern**

“Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.”

Let's say you're waiting for an important email from the manager. Once you receive it, you'll have to immediately make the required changes to the codebase. You could send them an email that says “Are you going to send it right now?” every 5 minutes. But they might and probably will be annoyed by it. The more natural way is to go on with your life and once they send the email, you'll have a notification and you can start working right away. This is what is called the observer pattern.